TECHNICAL INTERVIEW QUESTION BANK

Pure Technical Round — Languages, CS Fundamentals, Tools & Project Deep-Dives

Prepared for

PRUTHVIRAJ THORBOLE

Companion to the Full Interview Preparation Guide — Technical Round Only

What's in this document

This is a pure technical-round question bank covering every language, tool, and concept that appears on your resume — C, C++, Java, Python, OOP, DSA, DBMS/SQL, MySQL vs MongoDB, web technologies, AWS/cloud, Git, and the specific APIs and tools used in each of your 3 projects (Firebase, Google Cloud Vision, Vertex AI, Maps API, Power BI, Pandas/NumPy).

Technical interviewers commonly test fundamentals in isolation (separate from your projects) to verify you understand concepts, not just that you used a tool once. This document is built to match that pattern.

Priority order for limited time: Section 4 (OOP) + Section 5 (DSA) + Section 6 (DBMS/SQL) > Section 2-3 (Java/Python) > Section 12 (Project Deep-Dives) > rest.

Table of Contents

1. C and C++ Fundamentals.....4
2. Java.....5
3. Python.....6
4. Object-Oriented Programming (Language-Agnostic).....8
5. Data Structures & Algorithms.....9
6. DBMS & SQL Fundamentals.....11
7. MySQL, MongoDB, Firebase, Supabase — Database Comparisons.....13
8. Web Technologies (HTML, CSS, JavaScript, React).....14
9. Cloud Computing & AWS.....15
10. Git & GitHub.....17
11. Data Analytics Tools (Pandas, NumPy, Power BI, Tableau).....17
12. Project-Specific Technical Deep-Dive.....19
13. AI/ML & Generative AI Fundamentals.....22
14. Rapid-Fire Round (One-Line Answers).....24

1. C and C++ Fundamentals

Your resume lists C and CPP under Programming Languages. Even if Java/Python are your daily drivers, expect at least 2-3 quick fundamentals questions here to check your low-level/coursework foundation.

Q1. What is the difference between C and C++?

Answer: C is a procedural programming language with no built-in support for object-oriented concepts — code is organized around functions and structs. C++ is a superset of C that adds object-oriented programming (classes, objects, inheritance, polymorphism), along with features like function overloading, templates, exception handling, and the Standard Template Library (STL). In short, C++ lets you write in a structured/procedural style like C, or use OOP, depending on the problem.

Q2. What is a pointer in C/C++? Why are pointers important?

Answer: A pointer is a variable that stores the memory address of another variable, rather than a value directly. They're important because they allow direct memory manipulation, efficient array/string handling, dynamic memory allocation (using malloc/free in C or new/delete in C++), and passing large data structures to functions by reference instead of copying them, which saves memory and improves performance.

Q3. What is the difference between malloc() and new (or free() and delete)?

Answer: malloc() (C) allocates a raw block of memory and returns a void pointer, without calling any constructor — you must cast it to the correct type. new (C++) allocates memory and also calls the constructor for the object, returning a correctly typed pointer. Similarly, free() simply releases memory in C, while delete releases memory and calls the destructor in C++. Mixing them (e.g., using free() on memory allocated with new) leads to undefined behavior.

Q4. What is the difference between call by value and call by reference?

Answer: In call by value, a copy of the actual argument is passed to the function, so changes made inside the function don't affect the original variable. In call by reference, the function receives a reference (or address, via a pointer) to the original variable, so changes inside the function do affect the original. C primarily uses call by value, with pointers used to simulate call by reference; C++ additionally supports true call by reference using the '&' reference syntax in function parameters.

Q5. What is a structure (struct) in C, and how is it different from a class in C++?

Answer: A struct in C is a user-defined data type that groups different variables (of possibly different types) under one name, used purely to bundle data — it cannot contain functions/methods in standard C. A class in C++ can contain both data members and member functions, supports access specifiers (public, private, protected), and supports OOP features like inheritance. In C++, struct actually can hold functions too, and the only real difference from a class is that struct members default to public while class members default to private.

Q6. What is memory leak, and how can it occur in C/C++?

Answer: A memory leak happens when dynamically allocated memory (via malloc/new) is never freed (via free/delete) before the pointer referencing it goes out of scope or is reassigned, making that memory permanently unreachable and unusable for the rest of the program's execution. It commonly occurs when a programmer forgets to free memory after use, or when an exception/early return skips the cleanup code. In modern C++, smart pointers (like unique_ptr and shared_ptr) help prevent this by automatically freeing memory when it's no longer needed.

2. Java

Java is listed as a core language on your resume. Expect both conceptual questions and small code-tracing questions.

Q1. What are the main features of Java that make it different from C++?

Answer: Java is platform-independent — code compiles to bytecode that runs on the JVM (Java Virtual Machine), following the principle of 'write once, run anywhere,' unlike C++ which compiles to machine-specific code. Java has automatic garbage collection (no manual memory management like malloc/free), doesn't support multiple inheritance of classes (only through interfaces), has no pointers (for safety), and has built-in support for multithreading and exception handling as core language features.

Q2. Explain JVM, JRE, and JDK.

Answer: JVM (Java Virtual Machine) is the engine that actually executes Java bytecode — it provides platform independence by abstracting the underlying OS/hardware. JRE (Java Runtime Environment) includes the JVM plus the core libraries needed to run Java applications, but not the tools to develop them. JDK (Java Development Kit) includes the JRE plus development tools like the compiler (javac), debugger, and other utilities needed to write and compile Java code.

Q3. What is the difference between an abstract class and an interface in Java?

Answer: An abstract class can have both abstract methods (without implementation) and concrete methods (with implementation), can have instance variables, and supports single inheritance — a class can extend only one abstract class. An interface (traditionally) could only declare method signatures without implementation, though since Java 8, interfaces can have default and static methods with bodies. A class can implement multiple interfaces, which is how Java achieves a form of multiple inheritance. I'd choose an abstract class when classes share a common base implementation, and an interface when I just need to enforce a contract/capability across unrelated classes.

Q4. What is the difference between method overloading and method overriding?

Answer: Method overloading happens within the same class, where multiple methods share the same name but differ in parameter type, number, or order — it's resolved at compile time (static/early binding). Method overriding happens between a parent and child class, where the child class redefines a method with the exact same signature as the parent's — it's resolved at runtime (dynamic/late binding) based on the actual object type, which is what enables polymorphism.

Q5. What is the difference between 'String', 'StringBuilder', and 'StringBuffer' in Java?

Answer: String objects are immutable — once created, their value can't be changed; any modification creates a new String object, which can be inefficient for heavy string manipulation. StringBuilder is mutable and allows in-place modification, making it efficient for building/modifying strings, but it's not thread-safe. StringBuffer is also mutable like StringBuilder, but its methods are synchronized, making it thread-safe at the cost of slightly slower performance. For single-threaded string-heavy operations, I'd use StringBuilder; for multithreaded scenarios needing thread safety, StringBuffer.

Q6. Explain exception handling in Java — what is the difference between checked and unchecked exceptions?

Answer: Exception handling in Java uses try, catch, finally, and throw/throws to gracefully handle runtime errors instead of crashing the program. Checked exceptions (like IOException) are checked at compile time — the compiler forces you to either handle them with try-catch or declare them with 'throws' in the method signature. Unchecked exceptions (like NullPointerException or ArithmeticException) extend RuntimeException and aren't checked at compile time — they typically represent programming errors that surface at runtime.

Q7. What is the output of this code, and why?

Answer: Code: int x = 5; int y = x++ + ++x; System.out.println(y);

x++ (post-increment) uses the current value of x (5) in the expression, then increments x to 6. ++x (pre-increment) increments x to 7 first, then uses 7 in the expression. So y = 5 + 7 = 12. This tests whether you correctly distinguish pre- vs post-increment evaluation order, a very common Java trick question.

Q8. What is garbage collection in Java, and can you force it?

Answer: Garbage collection is the JVM's automatic process of identifying and reclaiming memory occupied by objects that are no longer reachable/referenced by the program, freeing the developer from manual memory management. You can request garbage collection using System.gc(), but this is only a suggestion to the JVM — it does not guarantee immediate or even eventual collection; the JVM decides when garbage collection actually runs based on its own internal heuristics.

3. Python

Python is your most heavily used language across internships and projects — expect deeper questioning here than on Java/C++.

Q1. What makes Python different from Java or C++ at a fundamental level?

Answer: Python is dynamically typed (variable types are determined at runtime, not declared explicitly) and interpreted (executed line-by-line by the Python interpreter, rather than compiled ahead of time to machine code), whereas Java and C++ are statically typed and compiled/JIT-compiled. This makes Python faster to write and more flexible, at some cost to raw execution speed and to catching type errors only at runtime instead of compile time.

Q2. What is the difference between a list, a tuple, and a dictionary in Python?

Answer: A list is an ordered, mutable collection that can hold duplicate values and is defined with square brackets — used when you need a changeable sequence. A tuple is also ordered but immutable — once created, it cannot be changed — and is typically used for fixed collections of values, like coordinates. A dictionary is an unordered (technically insertion-ordered since Python 3.7) collection of key-value pairs, used for fast lookups by key rather than by position.

Q3. What are *args and **kwargs in Python? Give an example of when you'd use them.

Answer: *args lets a function accept any number of positional arguments, collected into a tuple inside the function. **kwargs lets a function accept any number of keyword arguments, collected into a dictionary. I'd use them when building a flexible function whose exact number of inputs isn't known in advance — for example:

```
def process_data(*args, **kwargs):
    for value in args:
        print(value)
    for key, val in kwargs.items():
        print(key, val)
```

Q4. What is a Python decorator, and can you explain with a simple example?

Answer: A decorator is a function that takes another function as input, adds extra behavior around it, and returns a new function — without modifying the original function's code. It's commonly used for logging, timing, or access control. For example:

```
def my_decorator(func):
    def wrapper():
        print('Before function runs')
        func()
        print('After function runs')
```

```
    return wrapper

@my_decorator
def say_hello():
    print('Hello')

say_hello()
# Output: Before function runs / Hello / After function runs
```

Q5. What is the difference between deep copy and shallow copy in Python?

Answer: A shallow copy creates a new object, but if the original object contains nested objects (like a list of lists), the inner objects are still shared references — modifying a nested object in the copy affects the original too. A deep copy recursively copies all nested objects as well, so the copy is fully independent of the original. In Python, you'd use copy.copy() for a shallow copy and copy.deepcopy() for a deep copy, from the built-in copy module.

Q6. What are Python list comprehensions, and why are they preferred over loops in many cases?

Answer: A list comprehension is a concise way to create a list by applying an expression to each item in an iterable, optionally with a filtering condition, all in a single readable line. They're often preferred because they're more concise and, in many cases, faster than an equivalent explicit for loop, since the looping is implemented internally in optimized C code. For example, to get squares of even numbers:

```
squares = [x**2 for x in range(10) if x % 2 == 0]
# Equivalent loop:
squares = []
for x in range(10):
    if x % 2 == 0:
        squares.append(x**2)
```

Q7. What is the Global Interpreter Lock (GIL) in Python, and why does it matter?

Answer: The GIL is a mutex in CPython (the standard Python implementation) that allows only one thread to execute Python bytecode at a time, even on a multi-core machine. It matters because it means true parallel execution of CPU-bound Python code isn't possible using standard threading — multithreading in Python is mainly useful for I/O-bound tasks (like network requests) where threads spend time waiting rather than computing. For CPU-bound parallelism, you'd use the multiprocessing module instead, which runs separate processes, each with its own GIL.

Q8. What is the difference between Pandas Series and DataFrame, and how do you handle missing values in a DataFrame?

Answer: A Pandas Series is a one-dimensional labeled array, similar to a single column, while a DataFrame is a two-dimensional labeled data structure, like a table with rows and columns — essentially a collection of Series sharing the same index. To handle missing values, I'd typically use df.isnull().sum() to first identify how many missing values exist per column, then either drop them with df.dropna() if they're a small fraction of the data, or fill them with df.fillna() using a sensible value like the column mean, median, or a placeholder, depending on what's appropriate for that specific column.

Q9. What is the difference between '==' and 'is' in Python?

Answer: '==' checks for value equality — whether two variables hold equal values, even if they're different objects in memory. 'is' checks for identity — whether two variables point to the exact same object in memory. For example, two separate lists with identical elements would be '==' true but 'is' false, since they're stored as two distinct objects even though their contents match.

Q10. How does exception handling work in Python? What's the difference between except Exception and a bare except?

Answer: Python uses try, except, else, and finally blocks. Code that might raise an error goes in try; the corresponding except block catches and handles a specific exception type. 'except Exception' catches any exception that derives from the base Exception class (which covers almost all standard errors), while a bare 'except:' catches literally everything, including system-exiting exceptions like KeyboardInterrupt — which is considered bad practice because it can silently swallow critical signals like a user pressing Ctrl+C. I'd always prefer catching specific exception types, or at minimum 'except Exception', over a bare except.

4. Object-Oriented Programming (Language-Agnostic)

OOP is explicitly listed as a Core CS Concept on your resume. Interviewers ask these independent of which language you answer in — pick Java or Python examples confidently.

Q1. Explain the four pillars of OOP with a real-world or code example for each.

Answer: Encapsulation: bundling data and the methods that operate on it within a class, and restricting direct access using access modifiers (private/protected) — e.g., a BankAccount class keeps its balance private and only allows changes through deposit()/withdraw() methods.

Abstraction: hiding internal implementation details and exposing only what's necessary — e.g., calling car.start() without needing to know the internal combustion logic.

Inheritance: a class (child/subclass) acquiring properties and behavior from another class (parent/superclass) — e.g., a SavingsAccount class inheriting from a general Account class.

Polymorphism: the same method name behaving differently depending on the object calling it — e.g., a draw() method behaving differently for a Circle object versus a Square object, both inheriting from a Shape class.

Q2. What is the difference between compile-time (static) polymorphism and runtime (dynamic) polymorphism?

Answer: Compile-time polymorphism is achieved through method overloading — the compiler decides which method to call based on the method signature at compile time. Runtime polymorphism is achieved through method overriding combined with inheritance — the actual method that runs is determined at runtime based on the real object type (not the reference type), typically implemented internally via a mechanism like a virtual method table.

Q3. What is constructor overloading? Can a constructor be private?

Answer: Constructor overloading means a class can have multiple constructors with different parameter lists, allowing objects to be created in different ways depending on what data is available at creation time. Yes, a constructor can be private — this is commonly used in the Singleton design pattern, where you want to prevent external code from creating new instances directly and instead force object creation through a controlled static method.

Q4. What is the difference between composition and inheritance? When would you prefer one over the other?

Answer: Inheritance models an 'is-a' relationship — a SavingsAccount is an Account. Composition models a 'has-a' relationship — a Car has an Engine, where Engine is a separate object referenced inside Car, rather than Car extending Engine. I'd prefer composition when the relationship isn't a true hierarchy and you want more flexibility — composition is generally considered more flexible and less tightly coupled than inheritance, since you can swap out the composed object without restructuring the class hierarchy. The common guideline is 'favor composition over inheritance' unless there's a clear is-a relationship.

Q5. What is a constructor, and what is a destructor? Does Python have destructors?

Answer: A constructor is a special method automatically called when an object is created, typically used to initialize the object's state/attributes. A destructor is called when an object is destroyed/deallocated, typically used to release resources like file handles or network connections. Python does have a destructor-like method called __del__(), but because Python uses garbage collection, exactly when (or if) __del__() runs isn't always predictable, so explicit resource cleanup (e.g., using a 'with' statement / context manager) is generally preferred over relying on __del__().

Q6. What is method overriding's relationship with the '@Override' annotation in Java, or why isn't there an equivalent in Python?

Answer: In Java, @Override is an optional annotation placed above a method to tell the compiler 'this method is intended to override a parent method.' If the signature doesn't actually match a parent method (e.g., due to a typo), the compiler raises an error instead of silently creating a new, unrelated method — this is purely a compile-time safety check, not a functional requirement. Python doesn't have an equivalent because it's dynamically typed and doesn't have a compile-time checking phase in the same way; overriding simply happens automatically if a child class defines a method with the same name as the parent's.

5. Data Structures & Algorithms

Listed as 'Basic' on your resume — interviewers will calibrate to fresher level but still expect correctness on fundamentals and Big-O reasoning.

Q1. What is Big-O notation, and what's the time complexity of common operations on an array vs. a linked list?

Answer: Big-O notation describes the upper-bound growth rate of an algorithm's time or space requirements as input size increases, ignoring constant factors — it tells you how an algorithm scales, not its exact runtime. For an array: accessing an element by index is O(1), but inserting/deleting in the middle is O(n) because subsequent elements need to shift. For a linked list: accessing an element requires traversal, so it's O(n), but inserting/deleting at a known node (given a reference to it) is O(1) since it just involves updating pointers, with no shifting needed.

Q2. Explain how binary search works and write its time complexity. What's the precondition for using it?

Answer: Binary search works on a sorted array by repeatedly comparing the target value to the middle element: if equal, you're done; if the target is smaller, search the left half; if larger, search the right half. This halves the search space each time, giving O(log n) time complexity. The critical precondition is that the array must already be sorted — binary search doesn't work correctly on unsorted data.

Q3. What's the difference between a stack and a queue? Give a real-world example of each.

Answer: A stack follows LIFO (Last In, First Out) — the most recently added element is the first one removed, like a stack of plates where you take from the top. A queue follows FIFO (First In, First Out) — the first element added is the first removed, like a line of people waiting at a ticket counter. Stacks are commonly used for things like undo functionality or function call management (the call stack), while queues are used for task scheduling or breadth-first traversal.

Q4. What is recursion? Can you write a simple recursive function, e.g., to calculate factorial?

Answer: Recursion is when a function calls itself to solve smaller instances of the same problem, with a base case that stops the recursion to prevent infinite calls. For factorial:

```
def factorial(n):
    if n == 0 or n == 1:   # base case
        return 1
    return n * factorial(n - 1)   # recursive case
```

Q5. What is the difference between linear search and binary search? When would you use linear search even though it's slower?

Answer: Linear search checks each element one by one until it finds the target, with O(n) time complexity, and works on both sorted and unsorted data. Binary search is O(log n) but requires sorted data. I'd use linear search when the data isn't sorted and sorting it first wouldn't be worth the overhead — for example, searching through a small, one-time, unsorted list where the cost of sorting exceeds the cost of just scanning through it once.

Q6. Explain bubble sort and quicksort. What's the time complexity of each, and why is quicksort generally preferred?

Answer: Bubble sort repeatedly compares adjacent elements and swaps them if they're in the wrong order, 'bubbling' the largest unsorted element to its correct position each pass — it has $O(n^2)$ time complexity in the worst and average case, making it inefficient for large datasets. Quicksort picks a 'pivot' element, partitions the array so smaller elements go left and larger go right of the pivot, then recursively sorts each partition — it has an average time complexity of O(n log n), though its worst case is $O(n^2)$ with a poor pivot choice. Quicksort is generally preferred for large datasets because its average-case performance is significantly better, and in practice it has good cache locality and low overhead.

Q7. What is a hash table / hash map, and how does it achieve O(1) average lookup time?

Answer: A hash table stores key-value pairs and uses a hash function to convert each key into an index in an underlying array, where the corresponding value is stored. Because the hash function computes the index directly from the key (rather than searching for it), lookup, insertion, and deletion are O(1) on average. Collisions (two keys hashing to the same index) are handled using techniques like chaining (storing a list at that index) or open addressing (finding the next available slot) — and a poor hash function or too many collisions can degrade performance toward O(n) in the worst case.

Q8. What is a binary tree, and what's the difference between a binary tree and a binary search tree (BST)?

Answer: A binary tree is a hierarchical data structure where each node has at most two children, typically referred to as left and right. A binary search tree adds an ordering constraint: for every node, all values in its left subtree are smaller, and all values in its right subtree are larger, than the node itself. This ordering is what allows efficient O(log n) average-case search, insertion, and deletion in a balanced BST, similar in spirit to how binary search works on a sorted array.

Q9. What is the difference between BFS and DFS for graph/tree traversal?

Answer: BFS (Breadth-First Search) explores all neighbors at the current depth level before moving to nodes at the next depth level, typically implemented using a queue — it's useful for finding the shortest path in an unweighted graph. DFS (Depth-First Search) explores as far as possible along one branch before backtracking, typically implemented using a stack or recursion — it's useful for tasks like detecting cycles or exploring all possible paths.

6. DBMS & SQL Fundamentals

DBMS Fundamentals is explicitly listed, and you wrote 10+ SQL queries in your Sales Dashboard project — expect both theory and a query-writing exercise.

Q1. What is the difference between DBMS and RDBMS?

Answer: A DBMS (Database Management System) is software that manages data storage and retrieval generally, and doesn't necessarily enforce relationships between data or a tabular structure — file-based systems can technically fall under this broad definition. An RDBMS (Relational DBMS) specifically organizes data into related tables with rows and columns, enforces relationships through keys (primary/foreign), and supports SQL and ACID properties. MySQL is an example of an RDBMS, while some NoSQL stores are DBMS but not RDBMS.

Q2. What are ACID properties in DBMS?

Answer: ACID describes properties that guarantee reliable database transactions:

- Atomicity: a transaction either fully completes or fully fails — there's no partial execution left in an inconsistent state.
- Consistency: a transaction brings the database from one valid state to another, never violating defined rules/constraints.
- Isolation: concurrent transactions don't interfere with each other, as if they were executed sequentially.
- Durability: once a transaction is committed, it remains so permanently, even in the event of a system crash.

Q3. What is the difference between primary key, foreign key, and a unique key?

Answer: A primary key uniquely identifies each row in a table and cannot be NULL or duplicated — a table can have only one primary key (though it can be composite, spanning multiple columns). A foreign key is a column that references the primary key of another table, used to enforce a relationship between two tables. A unique key also enforces uniqueness like a primary key, but it can accept a NULL value (in most RDBMS, one NULL is typically allowed) and a table can have multiple unique keys, unlike having only one primary key.

Q4. Write a SQL query to find the second-highest salary from an Employees table.

Answer: There are multiple approaches; here's one using a subquery:

```
SELECT MAX(salary) AS second_highest_salary
FROM Employees
WHERE salary < (SELECT MAX(salary) FROM Employees);

-- Alternative using LIMIT/OFFSET (MySQL):
SELECT DISTINCT salary FROM Employees
ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

Q5. What is the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN?

Answer: INNER JOIN returns only the rows where there's a match in both tables. LEFT JOIN returns all rows from the left table, and matching rows from the right table — unmatched right-side columns return NULL. RIGHT JOIN is the mirror of LEFT JOIN, returning all rows from the right table. FULL OUTER JOIN returns all rows from both tables, with NULLs where there's no match on either side. In my Sales Data Analysis project, I'd typically use INNER JOIN when combining sales and product tables where every sale must correspond to a valid product, and LEFT JOIN if I wanted to include products even if they had zero sales.

Q6. What is the difference between WHERE and HAVING clauses in SQL?

Answer: WHERE filters individual rows before any grouping/aggregation happens, and cannot be used with aggregate functions like SUM() or COUNT() directly. HAVING filters groups after a GROUP BY aggregation has been applied, and is specifically designed to work with aggregate functions. For example, to find regions with total revenue over a threshold, I'd use GROUP BY region HAVING SUM(revenue) > 10000, not WHERE, since the filter depends on an aggregated value.

Q7. What is database normalization, and what problem does it solve? Can over-normalization cause issues?

Answer: Normalization is the process of organizing tables and relationships to minimize data redundancy and avoid update/insertion/deletion anomalies — for example, avoiding storing a customer's address repeated in every order row, which could lead to inconsistent data if it's updated in one place but not another. Yes, over-normalization can cause issues — splitting data into too many tables can require excessive JOINs for simple queries, hurting read performance, which is why some systems intentionally denormalize certain tables for reporting/analytics use cases, like a sales dashboard, where read speed matters more than minimizing redundancy.

Q8. What is an index in a database, and what's the trade-off of adding one?

Answer: An index is a data structure (often a B-tree) that allows the database to find rows matching a query condition much faster than scanning the entire table, similar in spirit to how a book's index lets you find a topic without reading every page. The trade-off is that indexes speed up read/SELECT queries but slow down write operations (INSERT, UPDATE, DELETE), since the index itself needs to be updated every time the underlying data changes, and indexes also consume additional storage.

Q9. What is a stored procedure, and have you used one?

Answer: A stored procedure is a precompiled set of SQL statements stored in the database that can be executed as a single callable unit, often used to encapsulate repeated business logic directly in the database layer. [Answer honestly — if you haven't used one in your projects:] I haven't used stored procedures directly in my projects so far — my SQL work has mainly involved writing standalone analytical queries for the Sales Dashboard project — but I understand the concept: they're useful for reducing repeated query logic and network round-trips, since the logic runs on the database server itself.

7. MySQL, MongoDB, Firebase, Supabase — Database Comparisons

You've used all four across different projects — expect a direct compare/contrast question, since using multiple tools well is a strong signal but only if you can articulate why each was chosen.

Q1. You've used MySQL, MongoDB, Firebase, and Supabase. Walk through when you'd choose each.

Answer: I'd frame the choice around the data shape and the project's needs:

- MySQL — relational, structured data with clear relationships and where data integrity/consistency matters most, like my Sales Data Analysis project with fixed columns like product, region, and revenue.
- MongoDB — flexible, document-based (JSON-like) data where the schema might evolve or vary between records, and where horizontal scalability matters more than strict relational integrity.
- Firebase — when I need real-time syncing across clients (e.g., live notifications/messaging) plus built-in authentication, ideal for fast-moving apps like FaithGuard where I didn't want to build my own auth and real-time infrastructure from scratch.
- Supabase — essentially an open-source alternative to Firebase, but built on top of PostgreSQL, so it gives me real-time features and built-in auth like Firebase, while still being a proper relational database under the hood if I need SQL querying power.

Q2. Supabase is built on PostgreSQL. What's the difference between PostgreSQL and MySQL?

Answer: Both are relational databases and share core SQL syntax, but PostgreSQL is generally considered more feature-rich for complex queries — it has stronger support for advanced data types (like JSON, arrays), more sophisticated indexing options, and stricter standards compliance. MySQL is historically known for being simpler to set up and very fast for straightforward read-heavy workloads, which made it popular for web applications. For most fresher-level or small-to-medium projects, the practical difference is small, but PostgreSQL tends to be preferred when the application needs more advanced querying capabilities.

Q3. What is the CAP theorem, and how does it relate to choosing between SQL and NoSQL databases?

Answer: The CAP theorem states that a distributed data system can only guarantee two out of three properties at the same time: Consistency (every read gets the most recent write), Availability (every request gets a response, even if not the latest data), and Partition tolerance (the system keeps working despite network failures between nodes). Traditional relational databases like MySQL typically prioritize consistency, while many NoSQL databases like MongoDB can be configured to favor availability and partition tolerance over strict consistency, which is part of why NoSQL systems are often chosen for large-scale, highly distributed applications where some temporary inconsistency is an acceptable trade-off.

Q4. How does Firebase Firestore handle real-time data syncing under the hood, conceptually?

Answer: Firestore maintains persistent, open connections (using WebSockets or similar long-lived connections) between the client and Firebase's servers. When a client subscribes/listens to a document or collection, any change to that data on the server is immediately pushed to all subscribed clients, rather than clients having to repeatedly poll the server asking 'has anything changed?'. This is what made it possible to implement near-instant message and notification delivery in FaithGuard without building custom WebSocket infrastructure myself.

8. Web Technologies (HTML, CSS, JavaScript, React)

Listed under Web Technologies, and React/JavaScript were used in FaithGuard — expect fundamentals plus a few React-specific questions.

Q1. What is the difference between HTML, CSS, and JavaScript, and how do they work together?

Answer: HTML defines the structure and content of a webpage (the elements present). CSS controls the presentation/styling — colors, layout, spacing. JavaScript adds behavior and interactivity — responding to user actions, manipulating the DOM dynamically, and making network requests. Together: HTML is the skeleton, CSS is the appearance, and JavaScript is the behavior that makes a static page interactive, like the real-time updates and form interactions in FaithGuard.

Q2. What is the DOM, and what does it mean to 'manipulate the DOM'?

Answer: The DOM (Document Object Model) is a tree-like, in-memory representation of the HTML page that the browser creates, where each HTML element becomes a 'node' that can be accessed and modified programmatically. 'Manipulating the DOM' means using JavaScript to dynamically add, remove, or change elements/content on the page after it has loaded — for instance, displaying a new notification banner in FaithGuard when Firebase pushes new data, without requiring a full page reload.

Q3. What is the difference between '==' and '===' in JavaScript?

Answer: '==' performs loose equality comparison, meaning it performs type coercion before comparing — so '5' == 5 evaluates to true because JavaScript converts the string to a number first. '===' performs strict equality comparison, checking both value and type without coercion — so '5' === 5 evaluates to false because one is a string and the other a number. Using '===' is generally considered best practice to avoid unexpected bugs from implicit type conversion.

Q4. What is a Progressive Web App (PWA), technically? What makes FaithGuard a PWA rather than just a website?

Answer: A PWA is a web application built using standard web technologies but enhanced with specific capabilities that make it behave more like a native app. The key technical requirements are: a Web App Manifest (a JSON file describing the app's name, icons, and how it should appear when installed) and a Service Worker (a JavaScript file that runs in the background, separate from the main page, enabling features like offline caching and push notifications). What makes FaithGuard a PWA rather than just a website is that it includes both of these — the manifest allows it to be 'installed' to a home screen, and the service worker enables offline support and the integration with Firebase Cloud Messaging for push notifications even when the app isn't actively open.

Q5. What is React, and what problem does it solve compared to plain JavaScript/DOM manipulation?

Answer: React is a JavaScript library for building user interfaces using a component-based architecture, where the UI is broken into reusable, self-contained pieces. It solves the problem of inefficient, hard-to-maintain direct DOM manipulation: instead of manually finding and updating DOM elements, React uses a virtual DOM — it calculates the minimal set of actual changes needed and updates only those parts of the real DOM, which is both more efficient and makes the code easier to reason about as the UI grows complex, like the multiple interactive modules in FaithGuard (reporting, browsing, messaging, notifications).

Q6. What are React hooks? Explain useState and useEffect.

Answer: Hooks are functions that let you use state and other React features inside functional components, without needing to write a class component. useState lets a functional component hold and update local state — calling it returns the current state value and a function to update it, and updating that state triggers a re-render. useEffect lets you run side effects (like fetching data, subscribing to a Firebase listener, or updating the DOM directly) after the component renders, and you can control when it re-runs using its dependency array. For example, in FaithGuard, I'd likely use useEffect to subscribe to a Firebase real-time listener when a component mounts, and clean up that subscription when it unmounts.

Q7. What is the difference between props and state in React?

Answer: Props (properties) are data passed into a component from its parent — they're read-only from the receiving component's perspective and are used to configure/customize a component from outside. State is data managed internally within a component itself, which can change over time (e.g., via useState) and triggers a re-render when updated. A simple rule: props flow down from parent to child and shouldn't be modified by the child; state belongs to and is controlled by the component itself.

9. Cloud Computing & AWS

Listed as 'Basic' under Cloud & DevOps — interviewers will calibrate questions to a foundational level but expect you to know what each service actually does.

Q1. What is cloud computing, in your own words, and what are the three main service models (IaaS, PaaS, SaaS)?

Answer: Cloud computing is the on-demand delivery of computing resources — servers, storage, databases, networking — over the internet, with pay-as-you-go pricing, instead of owning and maintaining physical hardware yourself.

- IaaS (Infrastructure as a Service): provides raw computing infrastructure like virtual machines, storage, and networking, where you manage the OS and above — e.g., AWS EC2.
- PaaS (Platform as a Service): provides a managed platform to deploy applications without managing the underlying servers — e.g., AWS Elastic Beanstalk.
- SaaS (Software as a Service): fully managed software delivered over the internet, where the user just uses the application — e.g., Gmail.

Q2. What is Amazon EC2, and what have you used it for?

Answer: EC2 (Elastic Compute Cloud) provides resizable virtual servers ('instances') in the cloud, where you choose the OS, compute power, and storage, and can scale up or down based on need, paying only for what you use. At a basic level, I've launched EC2 instances to host simple applications and test server configurations, choosing instance types and configuring basic security group rules to control inbound/outbound traffic.

Q3. What's the difference between Amazon S3 and Amazon RDS?

Answer: S3 (Simple Storage Service) is object storage, designed for storing and retrieving files of almost any type and size — like images, backups, or static website files — accessed via a key (essentially a path/filename), not through SQL queries. RDS (Relational Database Service) is a managed relational database service that runs database engines like MySQL or PostgreSQL for you, handling tasks like backups, patching, and scaling, so you interact with it using standard SQL rather than treating it as file storage.

Q4. What is IAM in AWS, and why is it important for security?

Answer: IAM (Identity and Access Management) is the AWS service used to control who can access which AWS resources and what actions they're allowed to perform — through users, groups, roles, and policies. It's important because it implements the principle of least privilege: each user or service should have only the permissions it

actually needs, nothing more, which limits the damage if credentials are ever compromised. I've used IAM at a basic level to create users and groups and attach specific permission policies, rather than giving broad admin access by default.

Q5. What is a VPC, and why would you need one?

Answer: A VPC (Virtual Private Cloud) is a logically isolated, private section of the AWS cloud where you can launch resources like EC2 instances within your own defined network — with control over IP address ranges, subnets, route tables, and security groups/firewalls. You need one because, by default, you don't want all your cloud resources openly exposed to the internet or to other AWS customers — a VPC lets you create private subnets for sensitive resources (like a database) that aren't directly internet-accessible, while still allowing controlled communication where needed.

Q6. What's the difference between a security group and a network ACL in AWS?

Answer: A security group acts as a virtual firewall at the instance level, controlling inbound and outbound traffic for individual EC2 instances, and is stateful — meaning if you allow inbound traffic on a port, the response traffic is automatically allowed out, without needing a separate outbound rule. A Network ACL (Access Control List) operates at the subnet level, affecting all instances within that subnet, and is stateless — meaning inbound and outbound rules must each be explicitly defined, since allowing one direction doesn't automatically allow the response.

Q7. How is AWS different from Google Cloud Platform (GCP), which you also use via Vision API and Vertex AI?

Answer: Both are major cloud providers offering largely overlapping categories of services — compute, storage, databases, AI/ML — but under different names and with different strengths. AWS has the largest overall market share and broadest range of services. GCP is often considered particularly strong in data analytics and AI/ML tooling (which is consistent with why I used GCP's Vision API and Vertex AI specifically for AI-related features in my projects, rather than AWS's equivalent services like Rekognition or SageMaker). At a basic level, the core cloud concepts — compute instances, object storage, managed databases, IAM — map conceptually between the two, even though the specific service names and APIs differ.

10. Git & GitHub

Used across your internship and all three projects — expect at least one practical question, not just definitions.

Q1. What is the difference between Git and GitHub?

Answer: Git is a distributed version control system — software that runs locally on your machine to track changes to your code over time. GitHub is a cloud-based hosting platform for Git repositories, which adds collaboration features on top of Git itself, like pull requests, issue tracking, and code review tools. You could use Git entirely locally without ever touching GitHub; GitHub just makes it easy to host, share, and collaborate on Git repositories remotely.

Q2. What is the difference between 'git pull' and 'git fetch'?

Answer: 'git fetch' downloads new commits and branch updates from the remote repository but does not merge them into your current local branch — it just updates your local knowledge of what exists remotely. 'git pull' does a fetch and then automatically merges the fetched changes into your current branch in one step. I'd use fetch when I want to review incoming changes before merging, and pull when I'm confident I just want to update my branch immediately.

Q3. What is a merge conflict, and how do you resolve one?

Answer: A merge conflict happens when Git can't automatically reconcile changes between two branches because the same lines of a file were modified differently in each — Git doesn't know which version is correct, so it pauses

the merge and marks the conflicting sections in the file using conflict markers (<<<<<<<, =======, >>>>>>>). To resolve it, I'd open the affected file, manually review both versions, decide on (or combine) the correct final content, remove the conflict markers, then stage and commit the resolved file to complete the merge.

Q4. What is the purpose of a .gitignore file?

Answer: A .gitignore file tells Git which files or folders to intentionally exclude from version control — commonly things like environment files containing secrets/API keys, build artifacts, dependency folders like node_modules, or IDE configuration files. This keeps the repository clean and prevents sensitive data (like Firebase API keys) or unnecessary large files from being accidentally committed and pushed to a public repository like GitHub.

Q5. What's the difference between 'git reset' and 'git revert'?

Answer: 'git reset' moves the current branch pointer to a previous commit, effectively undoing commits — depending on the mode (soft/mixed/hard), it can also alter the staging area and working directory, and it rewrites history, which is risky on shared/public branches. 'git revert' creates a new commit that undoes the changes introduced by a previous commit, without altering existing history — making it safer for shared branches since it doesn't rewrite anything that's already been pushed and pulled by collaborators.

11. Data Analytics Tools (Pandas, NumPy, Power BI, Tableau)

Directly tied to your Sales Data Analysis Dashboard project and your IBM Data Analyst certification.

Q1. What is the difference between Pandas and NumPy, and why are they often used together?

Answer: NumPy provides efficient, fixed-type, multi-dimensional array objects (ndarrays) and fast mathematical/numerical operations on them, optimized in compiled C code under the hood — it's the foundation for numerical computing in Python. Pandas builds on top of NumPy and adds labeled, table-like data structures (Series and DataFrame) along with tools specifically for data cleaning, filtering, grouping, and analysis. They're used together because Pandas internally relies on NumPy arrays for its underlying data storage and many numeric operations, so combining them gives both the convenience of labeled, tabular data handling and the performance of NumPy's optimized array operations.

Q2. How do you merge or join two DataFrames in Pandas? What's the equivalent of a SQL JOIN?

Answer: Pandas provides pd.merge(), which works very similarly to a SQL JOIN — you specify the two DataFrames, the column(s) to join on, and the join type (inner, left, right, outer):

```
merged_df = pd.merge(sales_df, products_df, on='product_id', how='inner')
# how can be 'inner', 'left', 'right', or 'outer' — mirrors SQL JOIN types
```

Q3. What is the groupby() function in Pandas, and how does it relate to SQL's GROUP BY?

Answer: groupby() splits a DataFrame into groups based on the values of one or more columns, then lets you apply an aggregate function (like sum, mean, count) to each group separately — conceptually identical to SQL's GROUP BY combined with an aggregate function. For example, to replicate a 'total revenue by region' query from my Sales Dashboard project:

```
revenue_by_region = df.groupby('region')['revenue'].sum().reset_index()
# SQL equivalent:
# SELECT region, SUM(revenue) FROM sales GROUP BY region;
```

Q4. What is DAX in Power BI, and how is it different from a SQL query?

Answer: DAX (Data Analysis Expressions) is the formula language used in Power BI to create calculated columns, measures, and custom aggregations directly within the Power BI data model. The key difference from SQL is context: SQL queries operate on raw rows in a database table independently, while DAX formulas are aware of the

report's filter context — meaning the same DAX measure can return different results depending on which filters/slicers are currently applied in the dashboard, since it recalculates dynamically based on what the user has selected.

Q5. What's the difference between a calculated column and a measure in Power BI?

Answer: A calculated column is computed row-by-row and physically stored in the data model, evaluated once when the data is loaded/refreshed — useful for categorizing or labeling each row. A measure is calculated on the fly based on the current filter context whenever it's used in a visual, and isn't stored as a static value — it's better for aggregations like total revenue or average order value, since these need to respond dynamically to whatever filters the user applies on the dashboard.

Q6. How do you decide which type of chart to use for a given dataset or insight — for example, in your Sales Dashboard?

Answer: It depends on what relationship I'm trying to show: a line chart for trends over time (like revenue trends month-over-month), a bar chart for comparing discrete categories (like revenue by product category or region), a pie chart sparingly for showing simple proportional breakdowns (though I try to avoid overusing pie charts since they can be harder to read accurately with many categories), and a scatter plot if I needed to show correlation between two numeric variables. In my Sales Dashboard, I used a line chart for revenue trends and bar charts for top-selling products, since those best matched the type of insight I was trying to surface.

12. Project-Specific Technical Deep-Dive

These go one level deeper than the general project questions in your main prep guide — pure implementation/technical detail an interviewer might press on.

12.1 FaithGuard

Q1. Technically, how is data structured in Firebase for FaithGuard — are you using Firestore or the Realtime Database, and why?

Answer: [State the actual one you used.] I used Firebase Firestore (or Realtime Database — specify which), which stores data as collections of documents, where each document is a JSON-like object. I structured it with separate collections for reports, messages, and notifications, each document referencing the anonymous user ID. I chose Firestore [if applicable] over the Realtime Database because it supports more expressive querying (filtering and ordering) and better scales for structured, growing collections like item reports, whereas the Realtime Database is a single large JSON tree, which becomes harder to query efficiently as data grows.

Q2. How do you secure Firestore/Firebase data so one anonymous user can't read or modify another user's reports or messages?

Answer: Firebase Security Rules control this at the database level — they're written declaratively and evaluated server-side on every read/write request, so they can't be bypassed by manipulating client-side code. For example, a rule would check that the requesting user's authenticated UID matches the 'ownerId' field on a document before allowing a write, and similarly restrict reads where appropriate (e.g., message threads should only be readable by the two participants involved).

Q3. What happens if a user loses internet connectivity while using FaithGuard as a PWA? Does anything still work?

Answer: Because it's a PWA with a service worker, certain functionality can still work offline — the service worker can cache previously loaded data and the app shell (UI/static assets), so the user could still browse previously fetched reports. However, any action requiring a live connection — submitting a new report, sending a message, or receiving real-time Firebase updates — would need to either fail gracefully with a clear offline message, or be

queued locally and synced automatically once connectivity returns, which is a common pattern for offline-first PWAs (though I'd clarify the exact level of offline support I implemented versus what would be an improvement area).

12.2 Cloud Image Recognition System

Q4. What format does the Google Cloud Vision API expect images in, and how did you handle different image formats/sizes in your Tkinter app?

Answer: Cloud Vision API accepts images either as base64-encoded content directly in the request, or as a reference to an image stored in Google Cloud Storage. In my application, when a user selects a local image file through Tkinter's file dialog, I read the file in binary mode and encode it to base64 before constructing the API request. For handling different formats, the Vision API supports common formats like JPEG, PNG, and BMP natively, so I didn't need custom format conversion in most cases, though very large images might need resizing/compression first to stay within API request size limits.

Q5. How do you handle errors from the Cloud Vision API — for example, if a request fails or the API quota is exceeded?

Answer: [Be honest about your actual error handling.] I wrapped each API call in a try-except block to catch exceptions like network failures or API errors, and displayed a user-friendly error message in the Tkinter interface rather than letting the app crash. For quota-exceeded errors specifically, the API typically returns a distinguishable error code/status, which I'd check for and handle separately — for example, showing a message suggesting the user try again later, rather than treating it the same as a generic failure.

Q6. Is the Tkinter application multi-threaded? What happens to the UI while waiting for the Cloud Vision API response?

Answer: [Answer based on actual implementation.] If implemented as a single-threaded application, the UI would freeze/become unresponsive while waiting for the API call to return, since Tkinter's main event loop is blocked during a synchronous network request. A better implementation — which I'd flag as an improvement if not already done — would run the API call in a separate thread (using Python's threading module) and use a callback or a thread-safe queue to update the Tkinter UI once the response arrives, keeping the interface responsive during the wait.

12.3 Sales Data Analysis Dashboard

Q7. Walk through the exact Pandas code you'd use to detect and remove duplicate rows, and to handle outliers in the revenue column.

Answer: For duplicates:

```
duplicate_count = df.duplicated().sum()
df = df.drop_duplicates()
```

Answer: For outliers, a common approach is using the IQR (Interquartile Range) method:

```
Q1 = df['revenue'].quantile(0.25)
Q3 = df['revenue'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
df_clean = df[(df['revenue'] >= lower_bound) & (df['revenue'] <= upper_bound)]
```

Q8. How does Power BI connect to and refresh data — did you use a static import, or a live/scheduled connection?

Answer: [State your actual setup.] I imported the cleaned dataset (likely as a CSV or Excel export from my Pandas cleaning step) directly into Power BI using Import mode, which loads a static snapshot of the data into Power BI's internal model. This is different from a Live Connection or DirectQuery mode, which would query the original data source in real time without storing a local copy, useful for large datasets that change frequently — but for a one-time analysis project like this, importing a cleaned, finalized dataset was the simpler and sufficient choice.

Q9. If your SQL queries and Power BI dashboard numbers ever showed different totals for the same metric, how would you debug that?

Answer: I'd first check whether both are pulling from the exact same underlying dataset/snapshot — a common cause of mismatches is comparing a SQL query run against the raw/original data versus a Power BI dashboard built on a cleaned or filtered version of that data. Next, I'd verify that any filters or slicers active in the Power BI report aren't silently narrowing the data being displayed. I'd also check for differences in aggregation logic — for example, whether NULL values are being excluded differently in the SQL query versus how Power BI's DAX measure handles them by default.

13. AI/ML & Generative AI Fundamentals

Tied to your Google AI-ML internship and Oracle Generative AI certification — expect conceptual questions rather than deep math, given fresher level.

Q1. What is the difference between Artificial Intelligence, Machine Learning, and Deep Learning?

Answer: AI is the broad concept of machines performing tasks that typically require human intelligence. Machine Learning is a subset of AI where systems learn patterns from data rather than being explicitly programmed with rules. Deep Learning is a subset of Machine Learning that uses multi-layered neural networks, particularly effective for complex, unstructured data like images, audio, and text — which is the technology underlying things like image recognition (relevant to my Cloud Vision project) and large language models.

Q2. What is overfitting in machine learning, and how can you prevent it?

Answer: Overfitting happens when a model learns the training data too specifically — including its noise and random fluctuations — so it performs very well on training data but poorly on new, unseen data, because it failed to learn the general underlying pattern. It can be prevented through techniques like using more training data, simplifying the model (reducing complexity/parameters), applying regularization (penalizing overly complex models), or using cross-validation to better estimate how the model performs on unseen data during training.

Q3. What is a confusion matrix, and what do precision and recall mean?

Answer: A confusion matrix is a table that summarizes a classification model's predictions against actual outcomes, broken into True Positives, True Negatives, False Positives, and False Negatives.

- Precision = True Positives / (True Positives + False Positives) — out of everything the model predicted as positive, how many were actually positive. Important when false positives are costly.
- Recall = True Positives / (True Positives + False Negatives) — out of all actual positives, how many did the model correctly catch. Important when missing a positive case is costly.

Q4. What is the transformer architecture, and why was it a breakthrough for language models?

Answer: The transformer architecture, introduced in 2017, processes all words/tokens in a sequence in parallel rather than one at a time sequentially (as older RNN-based models did), using a mechanism called self-attention to weigh how relevant every other word in the input is to understanding the current word. This parallelization made training dramatically faster and more scalable on large datasets, and the attention mechanism made it much better

at capturing long-range relationships between words — which is the foundation underlying modern large language models.

Q5. What is fine-tuning, and how is it different from prompt engineering?

Answer: Fine-tuning involves further training an existing pre-trained model on a smaller, task-specific dataset, actually updating the model's internal weights so it becomes specialized for that task. Prompt engineering, by contrast, doesn't change the model at all — it involves carefully crafting the input/instructions given to an already-trained model to guide it toward the desired output. Prompt engineering is faster and cheaper since no retraining is involved, while fine-tuning can achieve more consistent, specialized behavior but requires more data, compute, and expertise.

Q6. What is Vertex AI, and how is it different from just calling a pre-built API like Cloud Vision API?

Answer: Vertex AI is Google Cloud's unified machine learning platform that lets you build, train, deploy, and manage custom ML models, as well as access pre-trained foundation models — it's a broader platform for ML development, not just a single-purpose API. Cloud Vision API, by contrast, is a specific pre-built API focused only on image analysis tasks (OCR, object detection, face detection, etc.) — you call it without needing to train or manage any model yourself. In my FaithGuard project, I used Vertex AI specifically for [state the actual use, e.g., a classification task], whereas in my Cloud Image Recognition project, I used the simpler, purpose-built Vision API since I didn't need custom model training for that use case.

14. Rapid-Fire Round (One-Line Answers)

Many technical interviewers end with a quick round of short, definition-style questions to test breadth. Keep these answers to one or two sentences max.

Q1. What is an API key used for?

It authenticates and authorizes requests to an API, identifying which application/account is making the call, often also used for rate-limiting and billing tracking.

Q2. What is JSON, and why is it widely used?

JSON (JavaScript Object Notation) is a lightweight, human-readable data format for representing structured data as key-value pairs; it's widely used because it's language-independent and easy for both humans and machines to read/parse.

Q3. What is the difference between synchronous and asynchronous code?

Synchronous code executes line by line, blocking further execution until the current operation finishes; asynchronous code allows other operations to continue while waiting for a long-running task (like a network request) to complete.

Q4. What is REST, and what does 'stateless' mean in REST APIs?

REST is an architectural style for designing networked APIs using standard HTTP methods; 'stateless' means each request must contain all the information needed to process it — the server doesn't store any client session state between requests.

Q5. What is the difference between GET and POST HTTP methods?

GET retrieves data and should not change server state, with parameters typically passed in the URL; POST sends data to the server to create or update a resource, with data sent in the request body.

Q6. What is caching, and why is it useful?

Caching temporarily stores frequently accessed data closer to where it's needed (e.g., in memory) so future requests can be served faster, reducing redundant computation or repeated calls to a slower data source like a database or external API.

Q7. What is the difference between authentication and authorization?

Authentication verifies who a user is (e.g., logging in); authorization determines what an authenticated user is allowed to do or access.

Q8. What does CRUD stand for?

Create, Read, Update, Delete — the four basic operations applicable to most data-driven applications and database interactions.

Q9. What is version control, and why is it important in a team setting?

Version control tracks and manages changes to code over time, allowing multiple people to collaborate without overwriting each other's work, and enabling rollback to previous versions if something breaks.

Q10. What is the difference between a compiler and an interpreter?

A compiler translates the entire source code into machine code before execution, producing a separate executable; an interpreter translates and executes code line-by-line at runtime, without producing a separate compiled file.

Q11. What is multithreading?

Multithreading is the ability of a program to execute multiple threads (smaller units of a process) concurrently, allowing better utilization of CPU resources, especially for tasks that can run independently.

Q12. What is the difference between vertical scaling and horizontal scaling?

Vertical scaling means adding more resources (CPU/RAM) to an existing single machine; horizontal scaling means adding more machines/instances to distribute the load across multiple servers.

Q13. What is an environment variable, and why use one instead of hardcoding values like API keys?

An environment variable stores configuration values (like API keys or database URLs) outside the source code itself; this keeps sensitive data out of version control and makes it easy to change configuration between environments (development, production) without modifying code.

Q14. What is the difference between frontend and backend development?

Frontend development focuses on the user-facing part of an application — what users see and interact with directly (UI/UX); backend development focuses on server-side logic, databases, and APIs that the frontend communicates with.

Q15. What does 'full-stack' mean?

A full-stack developer works across both the frontend (client-facing UI) and backend (server-side logic, database, APIs) layers of an application, rather than specializing in only one side.

Final Notes

Prep Tip: Wherever you see [bracketed text], that's a spot where only you know the real implementation detail (e.g., your actual Power BI connection mode, or exact framework used at PrepGrad) — fill these in honestly before the interview.

Prep Tip: This document is a companion to your full Interview Preparation Guide — use that one for HR/behavioral and resume-narrative questions, and this one purely for technical-round drilling.

Prep Tip: If a technical interviewer asks something you genuinely don't know, say so directly and explain your reasoning/approach to figuring it out — that's almost always scored better than guessing confidently and being wrong.